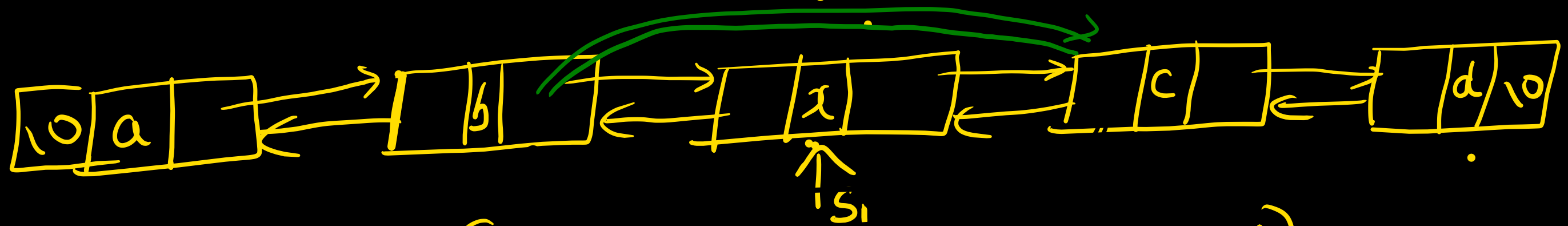


→ Delete a node with data x from a DLL



(3) Cases:

- 1) x is 1 node ✓
- 2) x is in last node ✓
- 3) x is in node between ✓

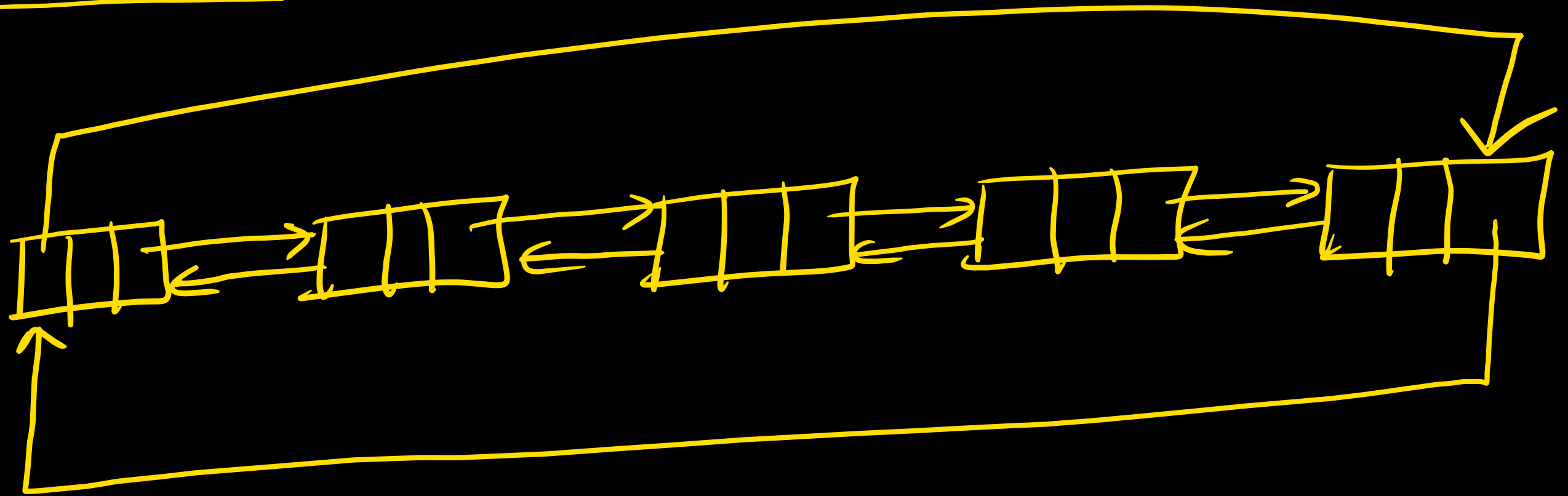
while ($S_1 \rightarrow \text{data} \neq x$ && $S_1 \rightarrow \text{next} \neq \text{null}$)
{
 $S_1 = S_1 \rightarrow \text{next}$ }
if ($S_1 \rightarrow \text{data} == x$)

$S_1 \rightarrow \text{prev} \rightarrow \text{next} = S_1 \rightarrow \text{next}$

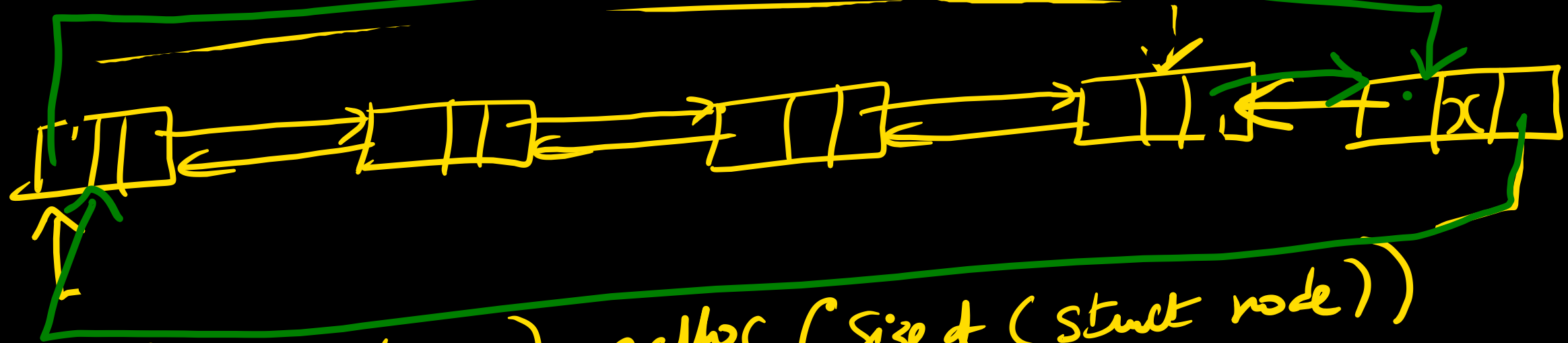
previous node should point to next node exists

modify the code.

Circular DLL



add a node with data 'x' at the end of CDLL



$p = (\text{struct node } *) \text{ malloc (Size of (struct node))}$
 $p \rightarrow \text{data} = x$

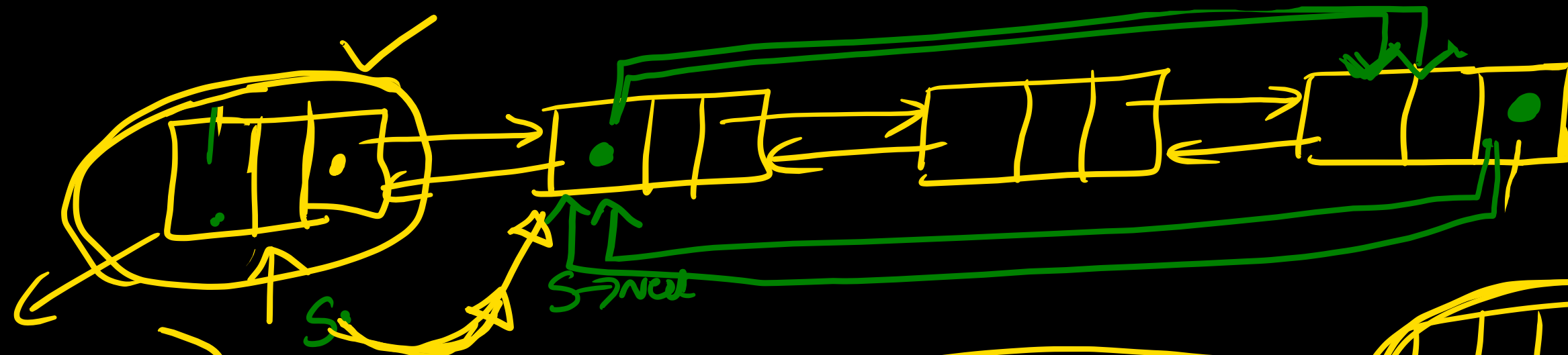
$p \rightarrow \text{next} = S$

$p \rightarrow \text{prev} = S \rightarrow \text{prev}$

$S \rightarrow \text{prev} = p$

$p \rightarrow \text{prev} \rightarrow \text{next} = p$

Delete 1 node in C-DLL



free
Storage is given back too



$S \rightarrow next \rightarrow prev = S \rightarrow prev$

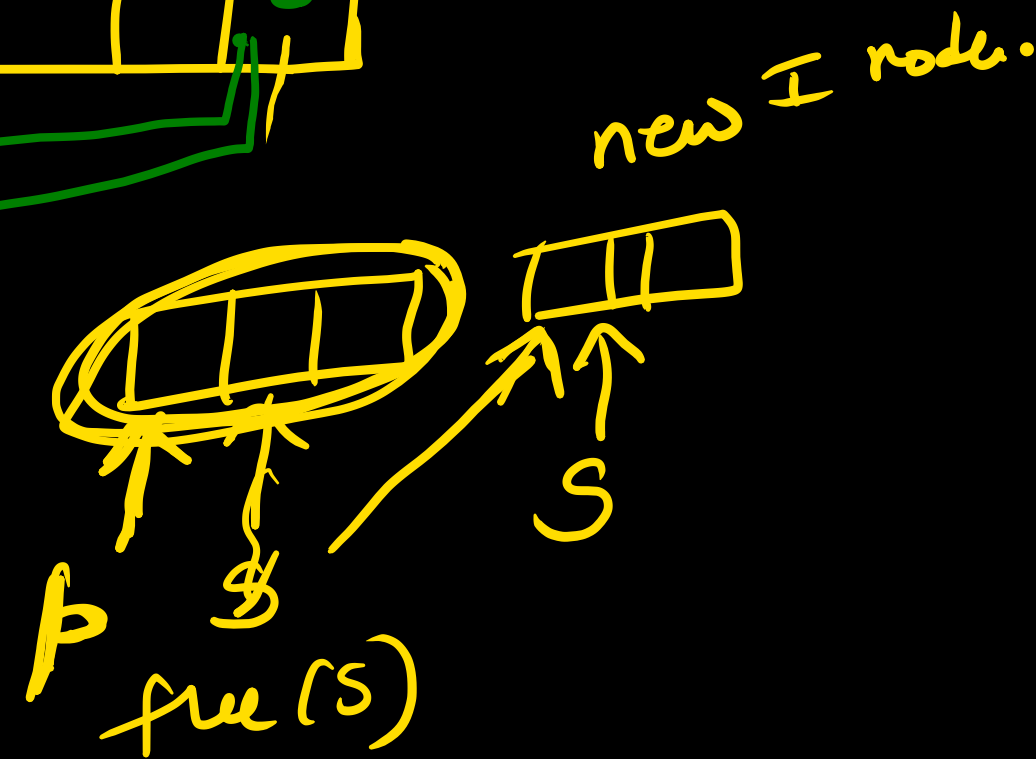
$S \rightarrow prev \rightarrow next = S \rightarrow next$

node * p = (S);

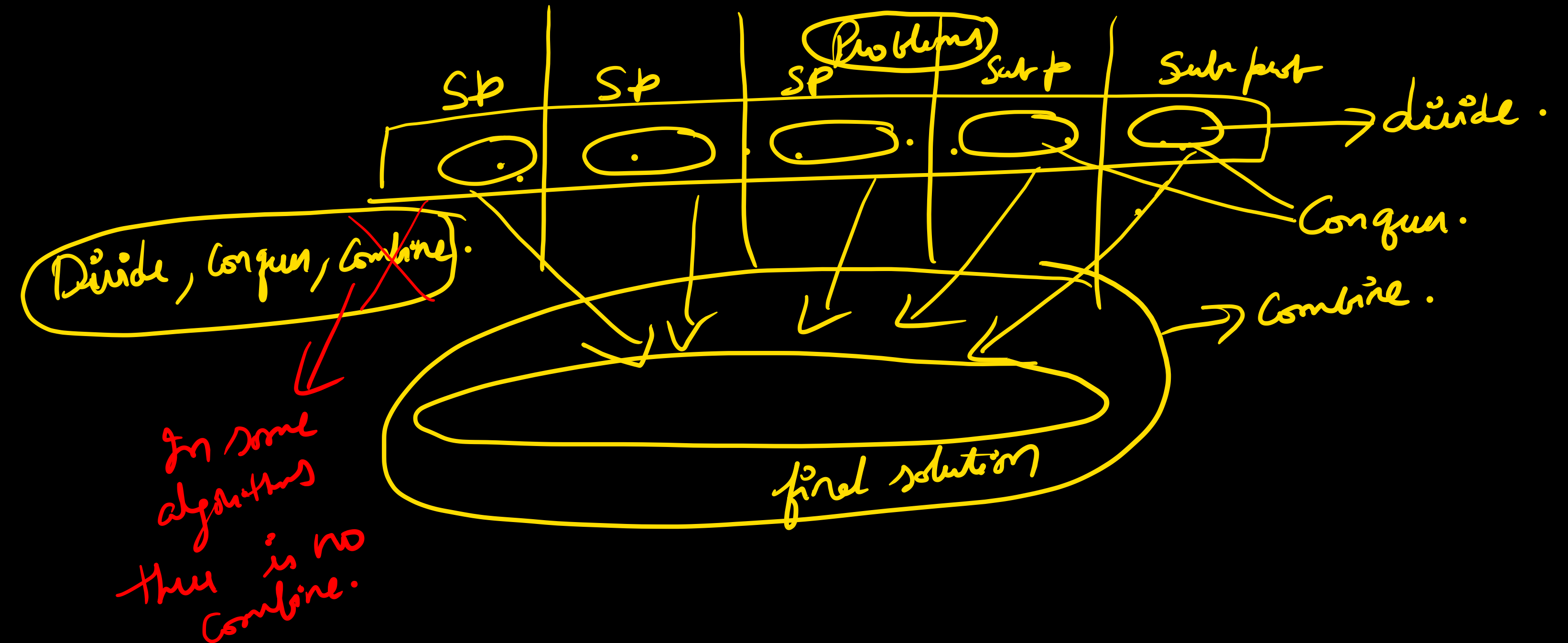
$S = S \rightarrow next;$

free(p)

dangling pointer.
 $p = \underline{\underline{null}};$



Divide and Conquer:-



Divide and Conquer:-

Given a function to compute on n inputs, the divide and Conquer suggests splitting the i/p into k distinct

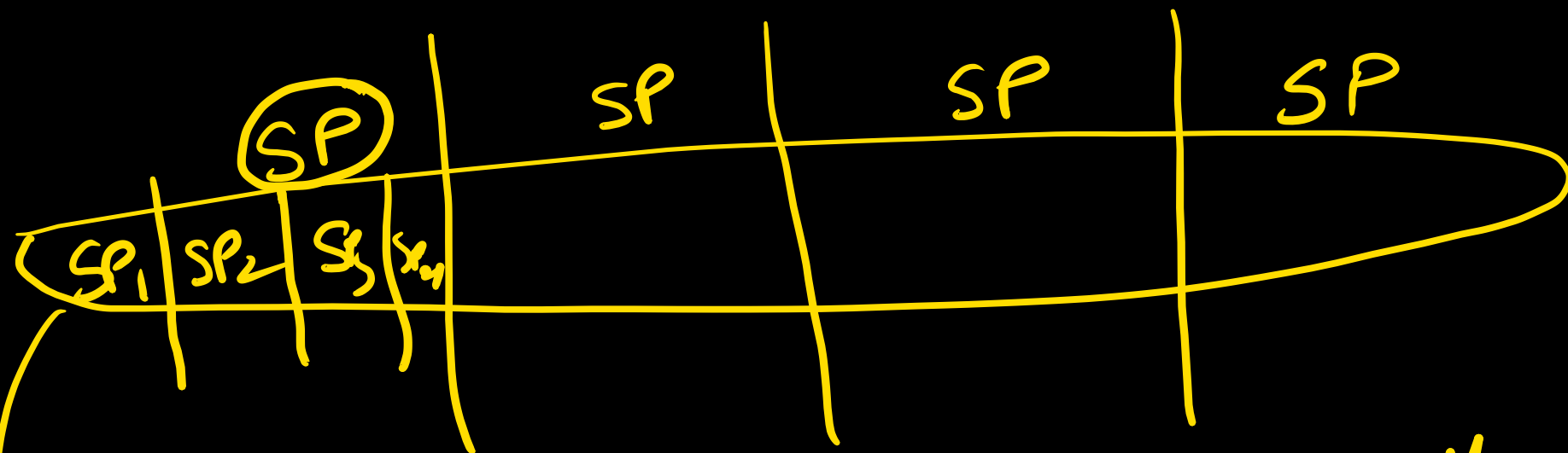
Sub problems, $1 < k \leq n$, yielding k sub problems.

These sub problems must be solved and a method must be found to combine sub solutions into a

solution of the whole. If the subproblems are still

relatively large, then divide and Conquer strategy

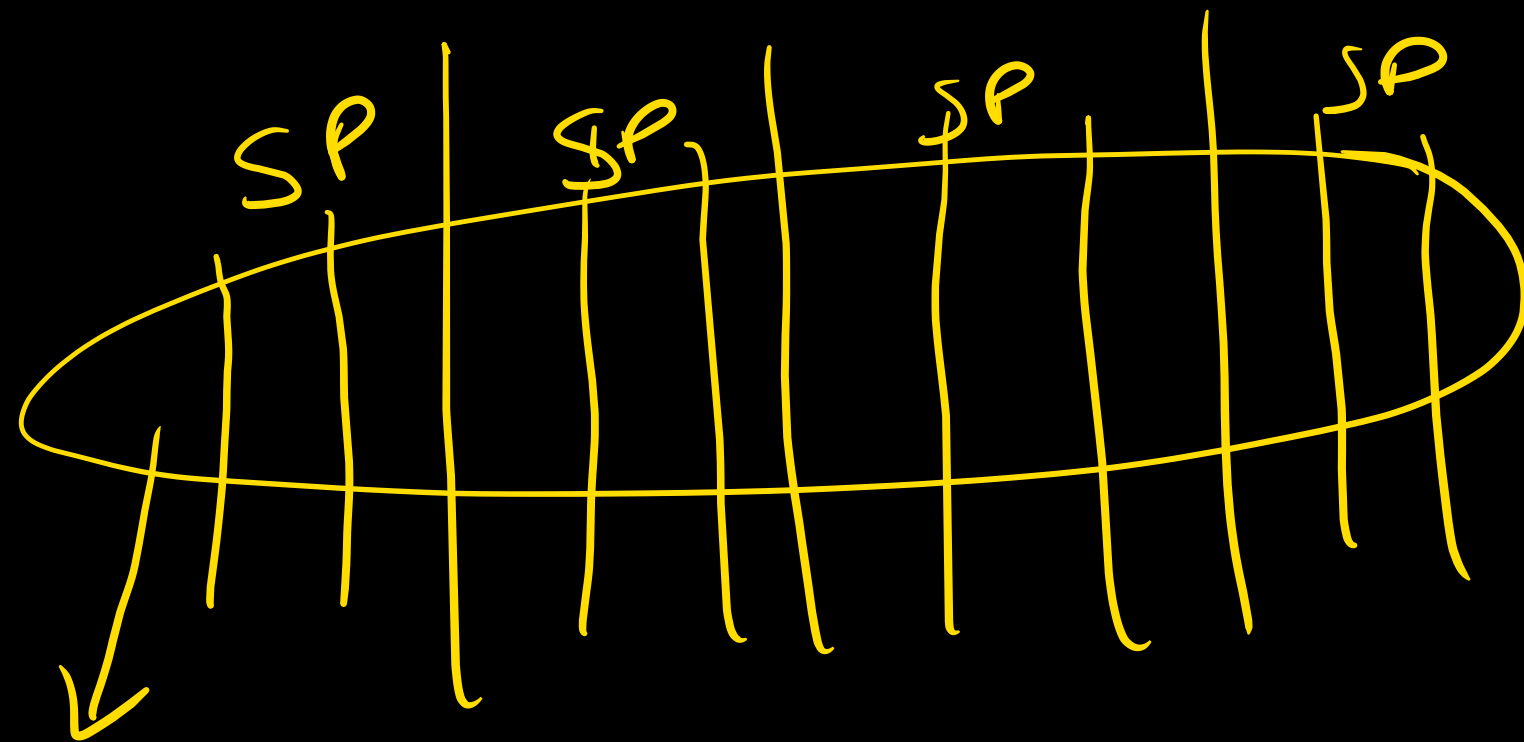
can be applied again



dividing sub problems into even smaller problems
 Can be done using recursion.

$\therefore$ D and C uses recursion

Division of a problem continues until it becomes
 the problem is small enough that can solved
 without splitting.



Small
problem
Can be
solved without
using further

Algorithm DAndC(P)

{ if small(P) then return S(P); ✓ → solution

else

{ divide 'P' into smaller instances

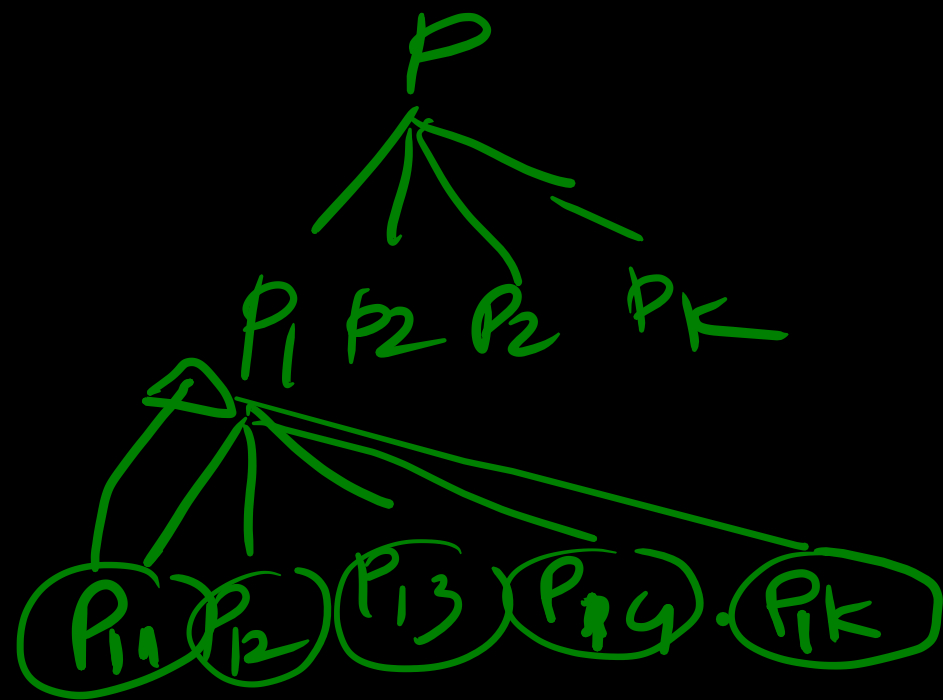
$P_1 P_2 P_3 \dots P_K$ $K \geq 1$

Apply DandC to each of these subproblems

return combine ($DandC(P_1)$, $DandC(P_2)$
 $\dots DandC(P_K)$);

}

}



Recursion

TC for D and C in recurrence relation

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{otherwise} \end{cases}$$

$g(n)$ is time for small problems

$f(n) \rightarrow$ is the time for
dividing and combining
solutions to subproblems

99%

99% of Danc problem $\rightarrow$ Some constant

$$T(n) = \begin{cases} T(1) & n=1 \\ a T(n/b) + f(n) & n>1 \end{cases}$$

where a and b are constants

and n is a power of b
i.e. $n = b^k$

ms:

$$T(n) = \underline{\underline{2T(n/2)}} + \underline{\underline{O(n)}} \checkmark$$

Sub
pec tree
in T.

BS

$$T(n) = T(n/2) + C$$

qs

$$T(n) = \underline{\underline{2T(n/2)}} + \underline{\underline{O(n)}} \text{ (avg case).}$$

Searching
 $x=15$



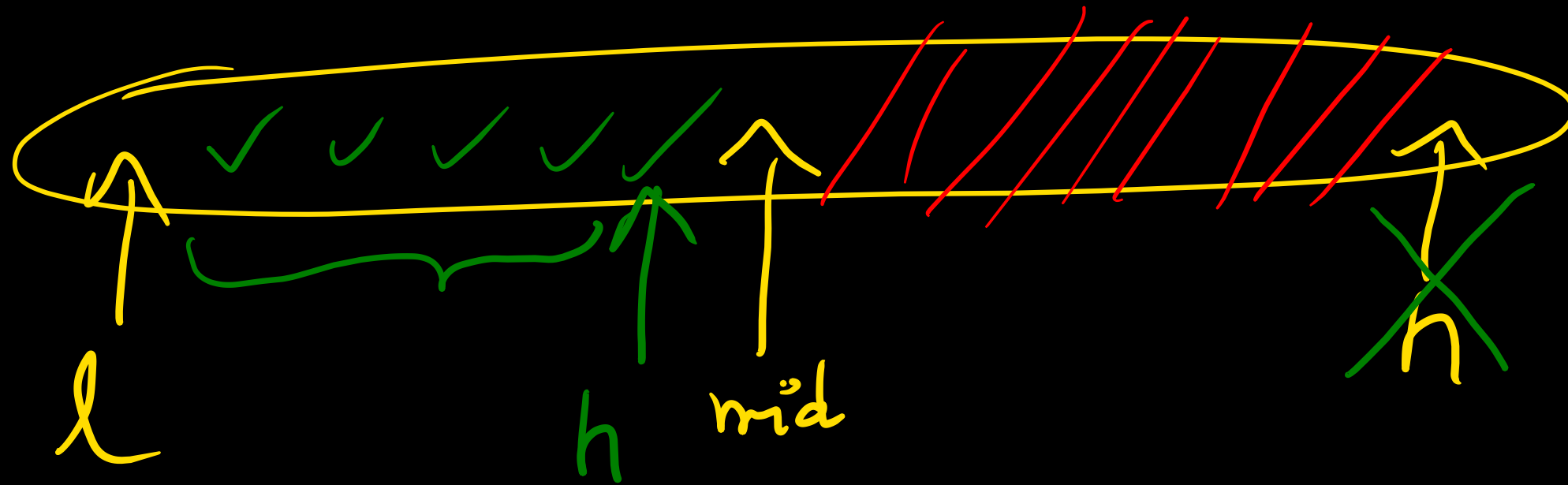
$$15 < 25$$

$$\text{high} \Rightarrow \text{mid} - 1$$

Binary search only works on sorted array.

$$\text{mid} = \frac{\text{low} + \text{high}}{2} = \left\lfloor \frac{1 + 8}{2} \right\rfloor = 4$$

$$\text{mid} = \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor = \left\lfloor \frac{1 + 3}{2} \right\rfloor = 2$$



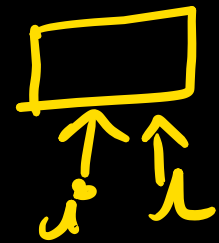
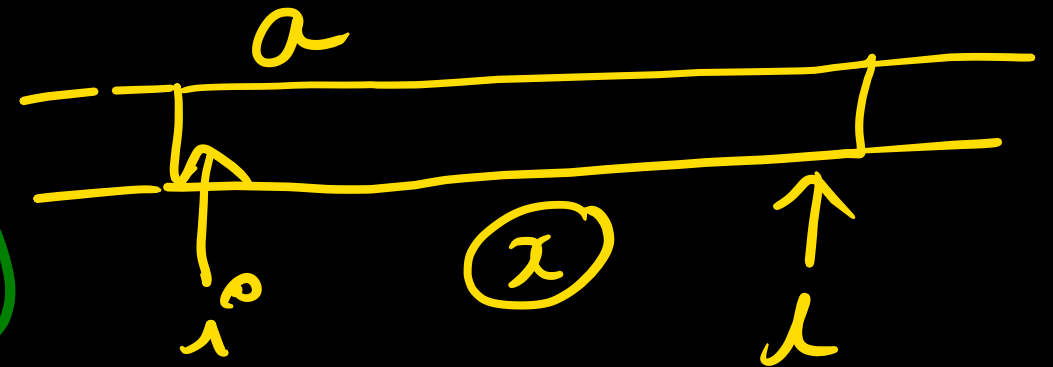
l to h is the search space.



Algo Bin Srch (a, i, l, x) ✓

→ only one element

$T(n)$



{ if ($l = i$) then

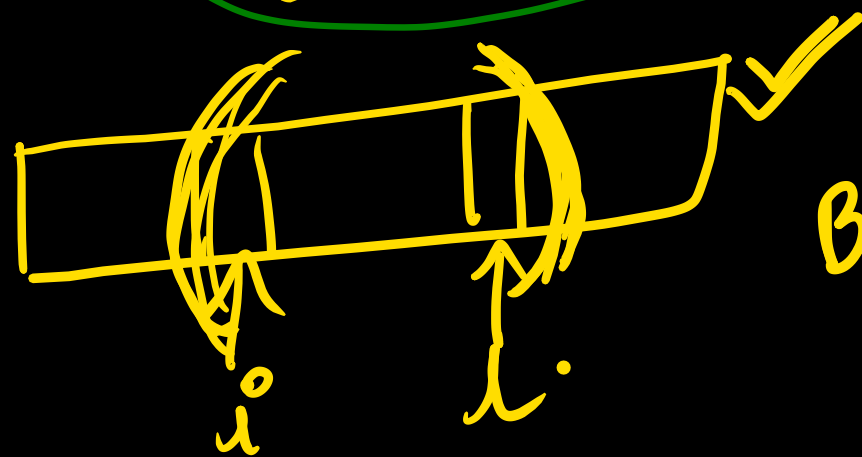
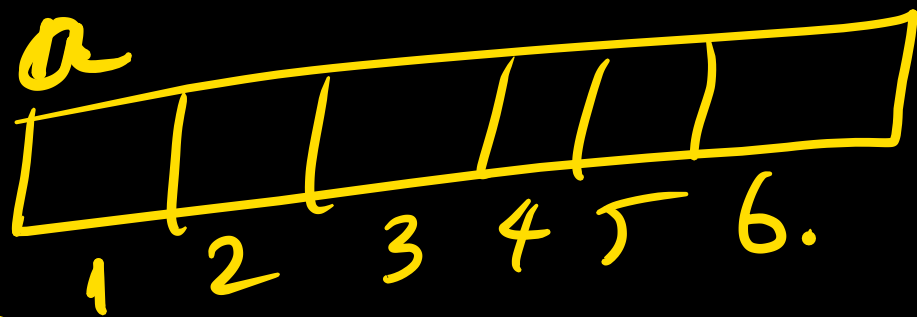
{ if ($x == a[i]$) then return i;

→ small problem

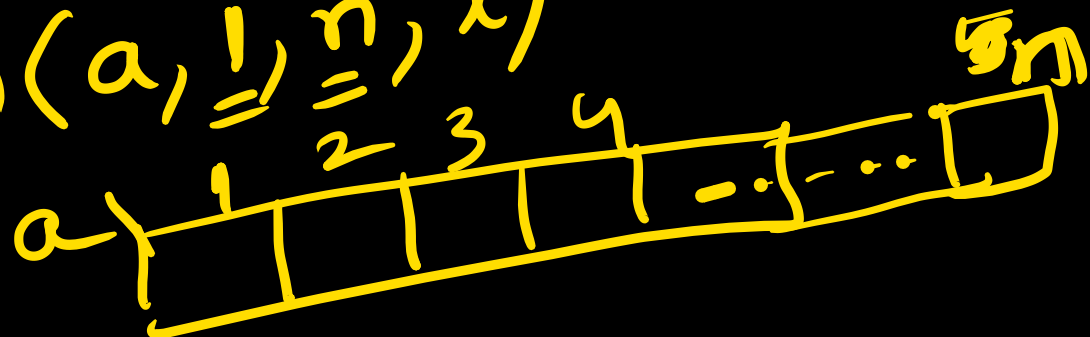
Ⓢ

else
return 0;

}



Bin Srch (a, 1, n, x)



else
{

$mid = \lfloor (i + l) / 2 \rfloor$; $\rightarrow$ ~~least~~ dividing the problem

if $x = a[mid]$ then return mid;

else if $(x < a[mid])$ then

return $BS(a, i, mid - 1, x)$

else
return $BS(a, mid + 1, l, x)$



$BS(a, i, l, x)$

$T(n/2)$ $\downarrow$ $mid - 1$

$T(n/2)$

}

$$T(n) = \begin{cases} c & ; n=1 \end{cases}$$

$$T(\underline{n}) = \begin{cases} \textcircled{a} & \text{when } n \leq 1 \\ & \downarrow \text{Comparing if } a[i] == x \\ T(n/2) + \textcircled{C} & \text{when } n > 1 \\ & \downarrow \text{dividing the problem} \end{cases}$$

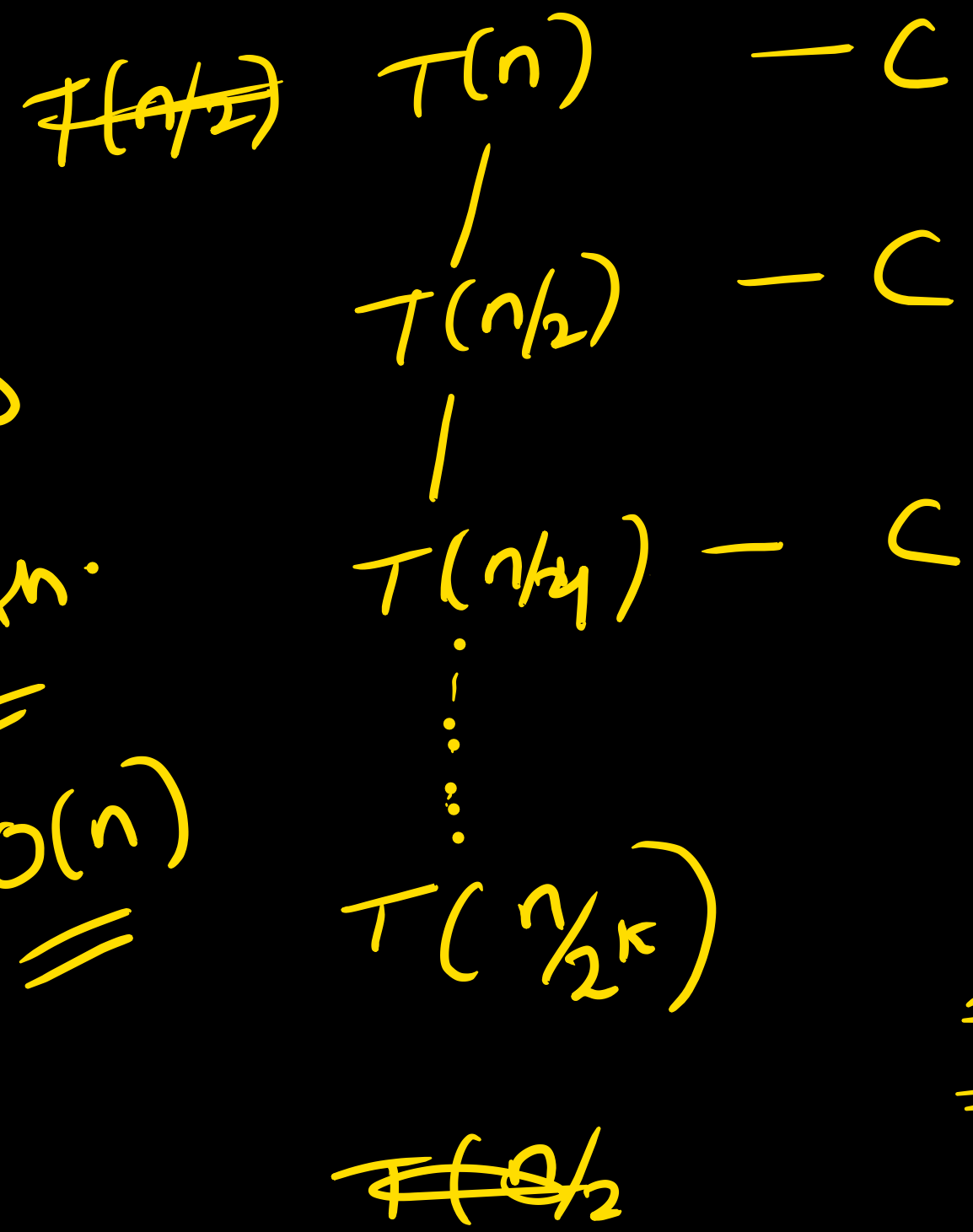
master theorem

$$= \begin{matrix} O(\log n) \\ n^{\log_2 a} \\ n^{\log_2 1} = \textcircled{1} \end{matrix} \quad \textcircled{\log n}.$$

$f(n)$
 $\textcircled{C}$

levels
of
depth.

$SC = O(n)$

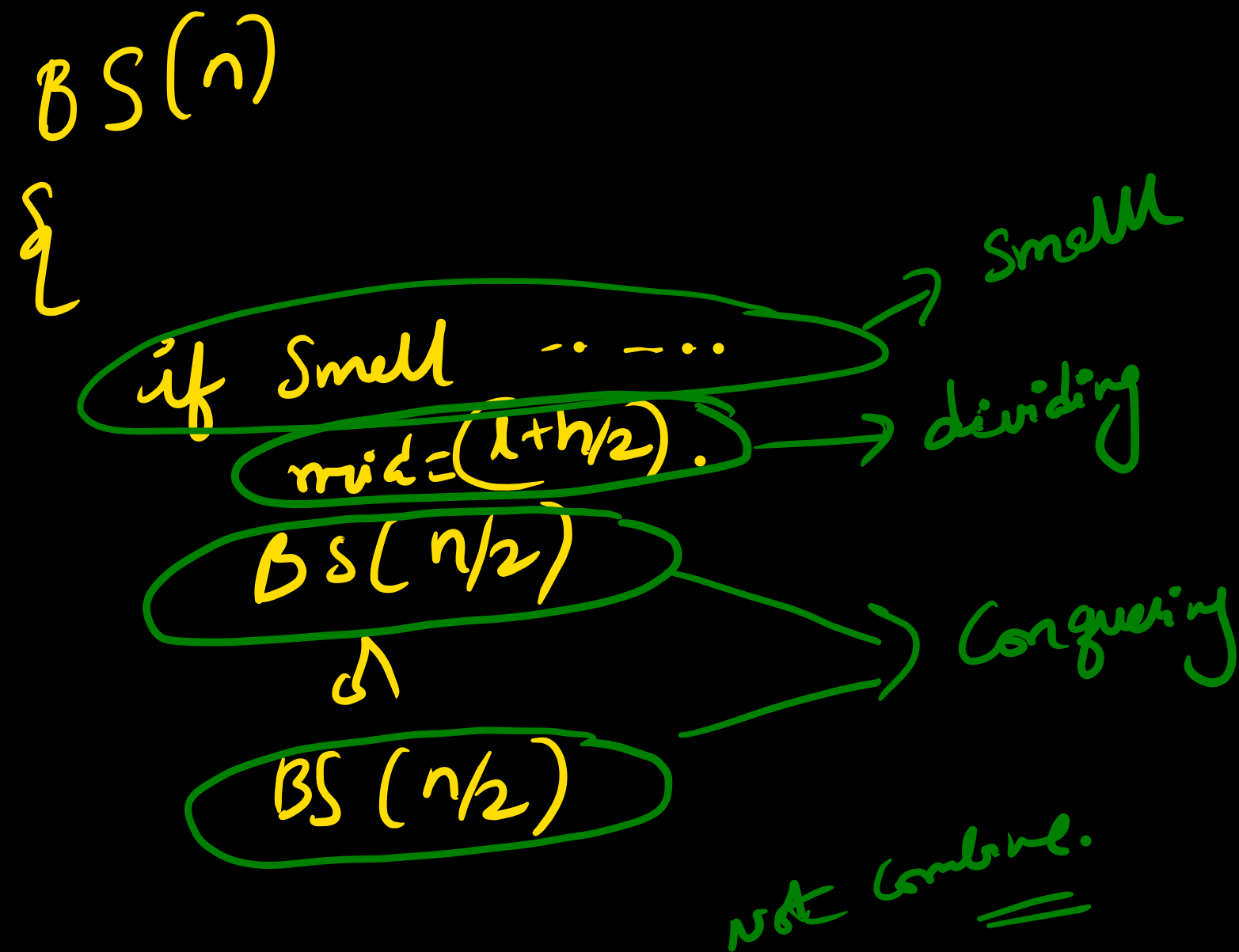
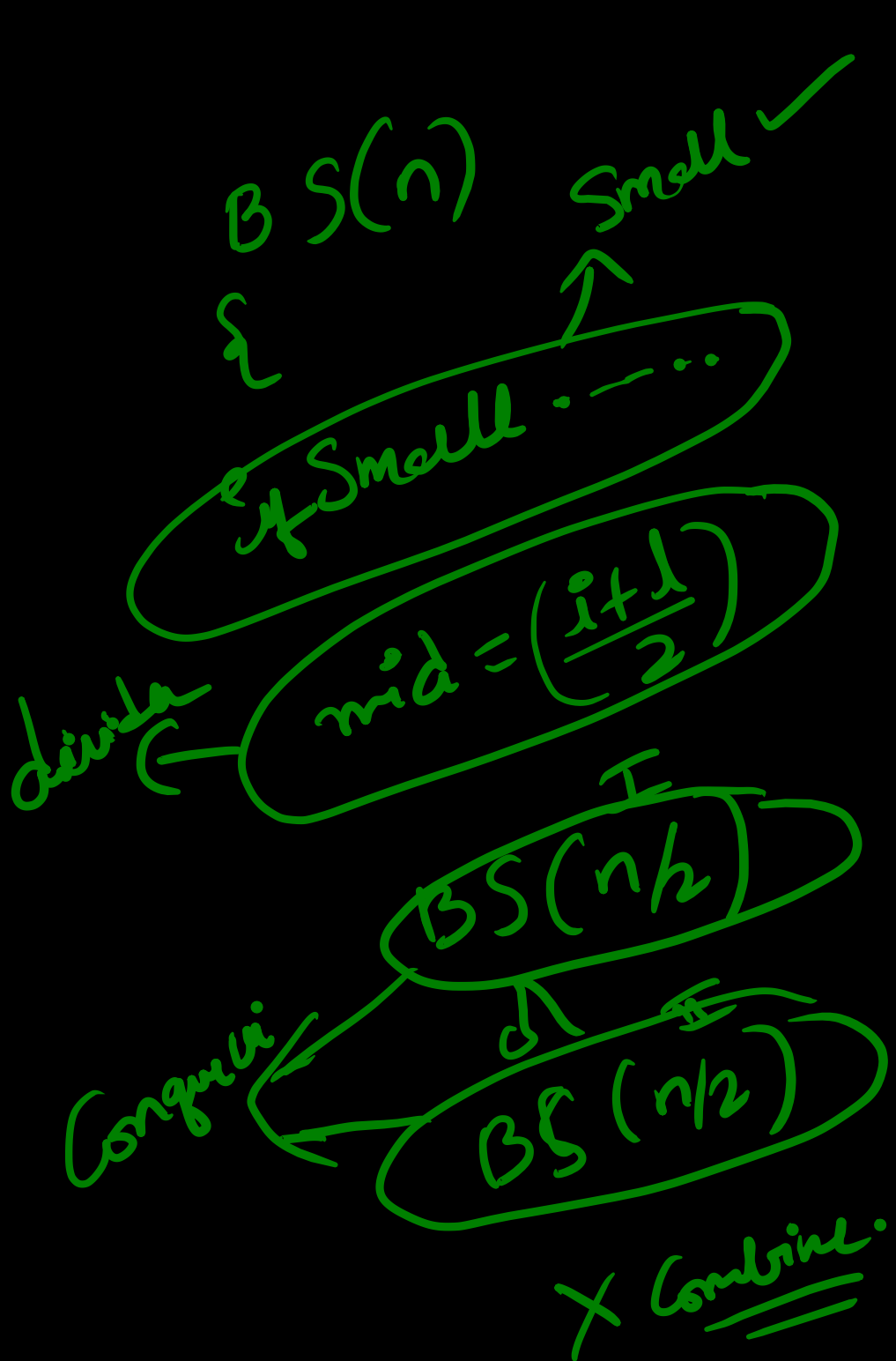
$$SC = \underline{\underline{O(n)}}$$


$K = ?$

$$\eta/2^k = 1$$
$$\Rightarrow k = \log \eta$$
$$\Rightarrow k = \log \eta$$

TC for BS for all cases is $O(\log n)$

SC for recursive BS is $O(\log n)$



iterative BS algo.

// → Comments

Algo Binsearch (a, n, x)

// Given an array a[1:n] of elements, search for x

✓ { low = 1 ; high = n ;

while (low ≤ high) do

✓ { mid = $\left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor$

if ($x < a[mid]$) then
high = mid - 1

else if ($x > a[mid]$)
low = mid + 1

else return mid;

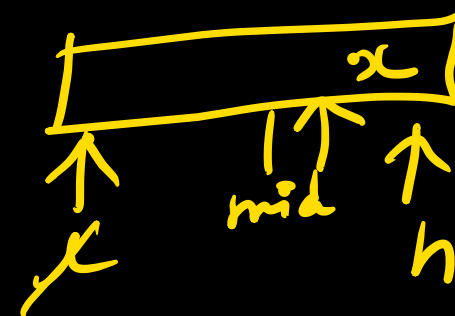
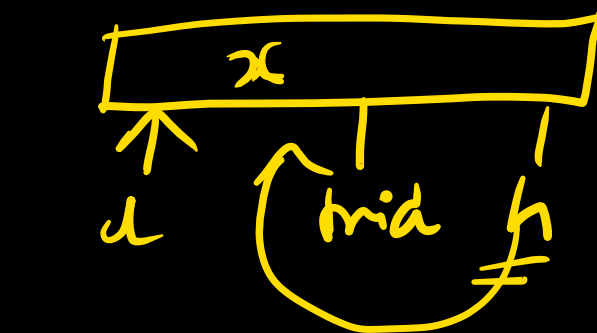
}

return 0

}

//

demo



Iterative:

$$TC = O(\log(n))$$

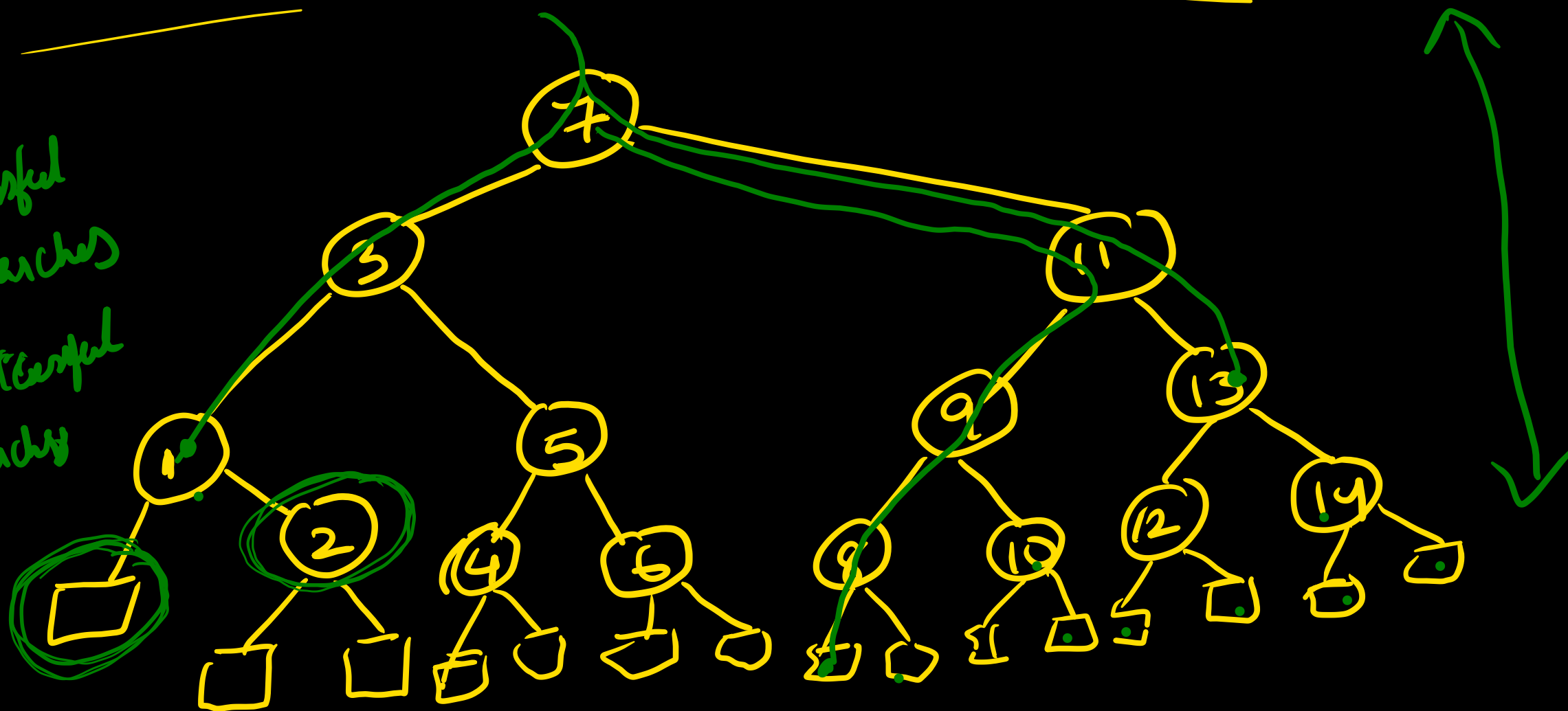
$$SC = O(1)$$

iterative is better than recursive.

1 2 3 4 5 6 7 8 9 10 11 12 13 14

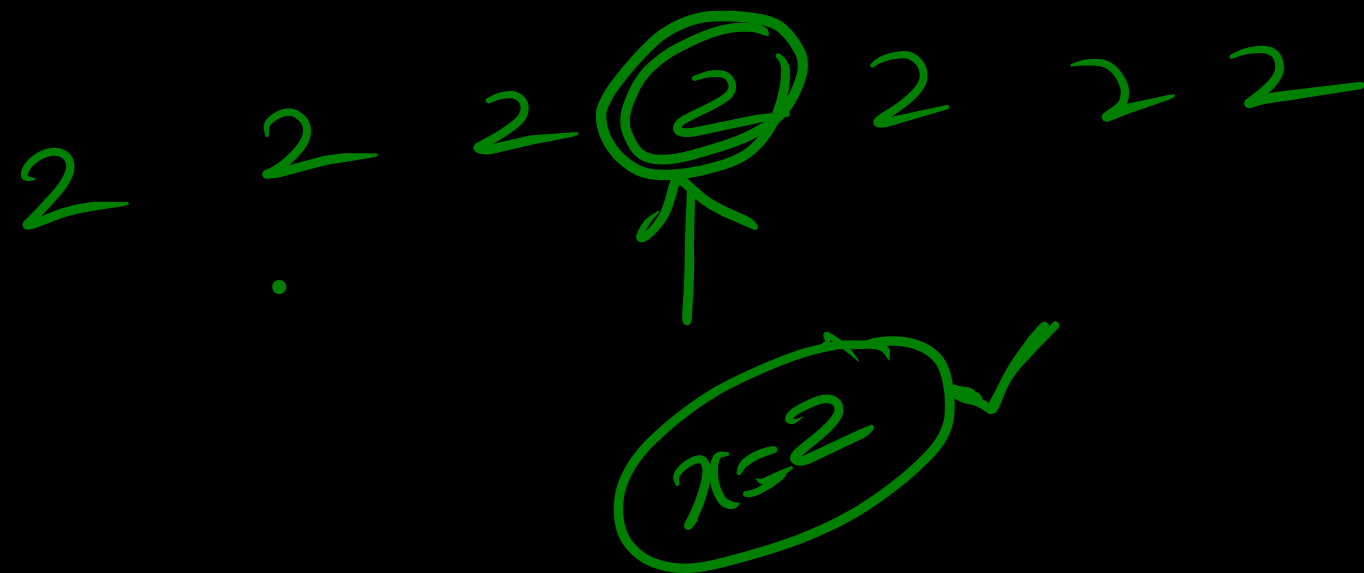
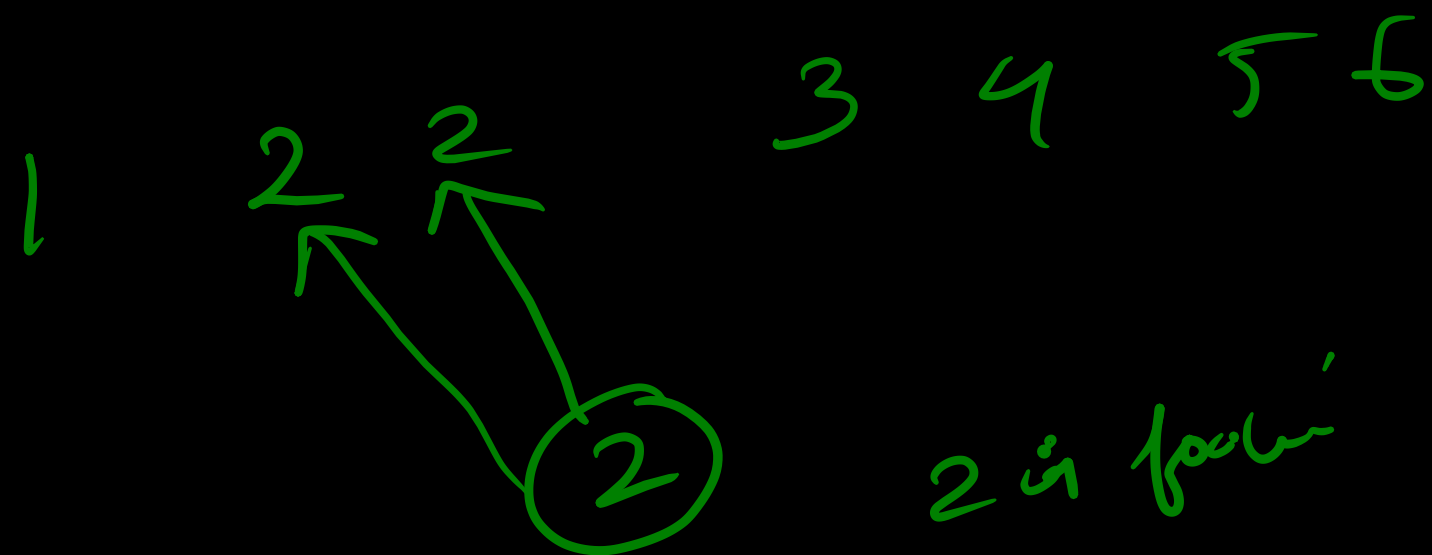
○ → Successful searches

□ → Unsuccessful searches



$O(\log n)$

Binary decision tree for binary search



Another application of O and C is min max:-

1	-1	2	16	10	19
---	----	---	----	----	----

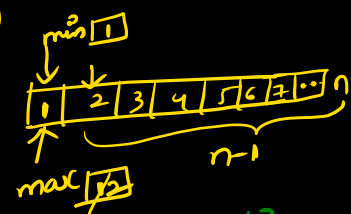
min = -1

max = 19

min max problem.

Algorithm straight max min (a, n, max, min)

{ max = min = a[1];



for (i = 2 to n do) (n-1) ✓
 { if a[i] > max { 1 comp }
 { if a[i] < min { 2 comp }
 { if (a[i] > max) then max = a[i];
 { else if (a[i] < min) then min = a[i];
 }
 } no of comparisons: 2(n-1)

max = 10
 min = 1
 a[i] = 100 > max ✓
 < min ✓

Best case i/p: always a[i] > max; next element should be greater than max computed so far



max = 10, 12, 14, 15 (n-1) ✓

min = 10

15, 14, 13, 12

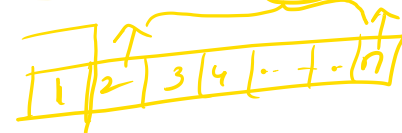
max = 15
 min = 15
 a[i] < max.
 a[i] < min.

2 comparisons

B.C: (n-1) w.C: 2(n-1) ✓

on average, a[i] is greater than max half of the time

$$1 \cdot \frac{(n-1)}{2} + 2 \cdot \frac{(n-1)}{2} = \frac{n-1}{2} + n-2$$



$$\left(\frac{3n}{2} - \frac{3}{2} \right) \checkmark$$